Wright State University

Computer Science and Engineering

CS 6900: Deep Learning

Project Submission

Project title: Cat vs Rabbit Image Classification Using ResNet18 and EfficientNet-B0

Team Name: Team FIG(Fire, Ice and Gold)

Team Members:

1.Aayush K.C.

2.Laxmi Satyal

3.Pramesh Maharjan

Contents

Objectives

The primary objective of this project is to design and evaluate a deep learning–based image classification system capable of distinguishing between images of cats and rabbits with high accuracy. To achieve this goal, the project focuses on the following specific objectives:

1. **To implement two advanced convolutional neural network (CNN) architectures—ResNet18 and EfficientNet-B0—for the binary classification task of identifying cats and rabbits.**
2. **To collect and prepare a Kaggle-based dataset consisting of cat and rabbit images**, applying preprocessing techniques such as resizing, normalization, and data augmentation. These steps aim to improve the robustness, diversity, and generalization capabilities of the models.
3. **To develop a fully supervised learning algorithm**, where each model is trained on labeled training data and evaluated using a separate testing set to determine real-world classification performance.
4. **To conduct a performance analysis** by comparing key metrics—such as accuracy, loss, model complexity, and computational efficiency—between ResNet18 and EfficientNet-B0.
5. **To identify the strengths and limitations of each architecture** in the context of small-scale binary image classification and understand how model design influences performance.
6. **To determine the model that demonstrates superior accuracy and effectiveness** for classifying cat and rabbit images and provide insights for potential improvements and future extensions of the project.

Description

This project aims to build an image classification system that can tell apart images of cats and rabbits using deep learning. It uses and compares two popular convolutional neural network (CNN) models: **ResNet18** and **EfficientNet-B0**. Both models are fine-tuned on a Kaggle dataset with labeled images of cats and rabbits.

The dataset goes through several image transformations, such as resizing the models, normalization, and data augmentation like horizontal flipping and rotation. These steps help the models become more robust and avoid model overfitting. The data is split into training and testing sets in an 80/20 ratio, so both models are tested fairly on new images after training.

Each model starts with pretrained ImageNet weights and is adjusted to classify two categories. The models use supervised learning, learning directly from labeled data. During training, metrics like training loss, validation loss, accuracy, precision, and recall are tracked to see how well each model works.

After training, the models are compared to find out which one gives better accuracy and generalization for this two-class task. The process covers data loading, setting up the models, training, real-time evaluation, and saving the best model.

In summary, this project shows a full workflow for image classification with modern deep learning models and highlights the strengths of both ResNet18 and EfficientNet-B0.

Selection of Models

In this project, ResNet18 and EfficientNet-B0 were selected as the two deep learning architectures for classifying images of cats and rabbits. Both models are well-established in the field of computer vision and provide strong baseline performance. However, they differ significantly in design philosophy, depth, parameter count, and computational efficiency. This contrast makes them ideal candidates for comparative evaluation on a relatively small, two-class dataset.

Reasons for Selecting These Models

1. ResNet18

- **Proven performance:** ResNet models consistently show high accuracy on image classification benchmarks such as ImageNet.
- **Residual connections:** Skip connections in ResNet help train deeper architectures by preventing vanishing gradients.
- **Lightweight backbone:** ResNet18 is one of the lighter variants, making it efficient for smaller datasets and faster to train.
- **Strong feature extraction:** Its convolutional structure captures key spatial features, making it suitable for tasks like distinguishing animal types.

2. EfficientNet-B0

- **High accuracy with fewer parameters:** EfficientNet-B0 achieves excellent performance while being much smaller and faster than many traditional CNNs.
- **Compound scaling strategy:** EfficientNet scales depth, width, and resolution in a balanced way, resulting in efficient computation.
- **Modern architecture:** EfficientNet-B0 is widely used for real-world applications because of its balance of speed and accuracy.
- **Good baseline for small and medium datasets:** Its lightweight structure makes it ideal for tasks with limited data or resources.

ResNet18 – Architecture and Parameters

ResNet18 is a convolutional neural network with 18 layers. It uses residual, or skip, connections to make training easier and prevent vanishing gradients.

Architecture Structure

- Input layer: 7x7 convolution with 64 filters, followed by max-pooling
- Stage 1: 2 residual blocks (each block has two 3x3 conv layers, total 4 conv layers)
- Stage 2: 2 residual blocks (4 conv layers)
- Stage 3: 2 residual blocks (4 conv layers)
- Stage 4: 2 residual blocks (4 conv layers)
- Global average pooling layer
- Fully connected (FC) layer with 2 output units (cat, rabbit)

Number of Layers

Total weighted layers:

- 17 convolution layers
- 1 fully connected layer
- Total = 18 layers.

Mathematical Description

A convolution layer computes:

$Y(i, j, k) = \text{sum over } c \left[(W(k, c) * X(c)) (i, j) \right] + \text{bias}(k)$

Where W is the filter and X is the input feature map.

A residual block computes:

$F(x) = \text{ReLU}(\text{Conv2}(\text{ReLU}(\text{Conv1}(x))))$

The final output of a block is:

$y = F(x) + x$

This skip connection helps improve gradient flow.

Parameter Calculation (Example)

Conv1: 7×7×3×64 + 64 = 9,472 parameters

Each BasicBlock in Stage 1 (64→64 channels):

Conv1: 3×3×64×64

Conv2: 3×3×64×64

Total per block ≈ 147,584 parameters

Total Parameter Count

Total parameters in ResNet18 ≈ 11.7 million.

EfficientNet-B0 – Architecture and Parameters

EfficientNet-B0 is a lightweight convolutional neural network. It uses MBConv blocks, which stand for Mobile Inverted Bottleneck Convolution, along with Squeeze-and-Excitation modules. The model uses compound scaling to balance its depth, width, and resolution.

Architecture Structure

EfficientNet-B0 has several stages, listed here in a format that works well in Word.

1. Stem: 3×3 convolution (input → 32 channels)
2. MBConv1 block (32 → 16 channels), 1 repetition
3. MBConv6 block (16 → 24 channels), repeated 2 times
4. MBConv6 block (24 → 40 channels), repeated 2 times
5. MBConv6 block (40 → 80 channels), repeated 3 times
6. MBConv6 block (80 → 112 channels), repeated 3 times
7. MBConv6 block (112 → 192 channels), repeated 4 times
8. MBConv6 block (192 → 320 channels), 1 repetition
9. Head: 1×1 convolution (320 → 1280 channels)
10. Fully connected layer (1280 → 2 classes)

Number of Layers

EfficientNet-B0 has about 237 layers in total. This count includes all convolution, depthwise, expansion, SE, and projection layers inside each MBConv block.

Mathematical Description

The MBConv block has three main steps:

1. Expansion:
2. X_e = ReLU6(W_e × X)
3. (expands channels, usually by 6×)

4. Depthwise convolution:
5. X_d = ReLU6(W_d ⊙ X_e)
6. (⊙ indicates channel-wise convolution)
7. Squeeze-and-Excitation:
8. z_c = average of X_d over spatial dimensions
9. s = sigmoid(W2 × ReLU(W1 × z_c))
10. X_s = s × X_d
11. Projection:
12. Y = W_p × X_s
13. Skip connection (if input and output shapes match):
14. Output = X + Y

Parameter Calculation

For an MBConv6 block (example: 24 → 40 channels):

- Expansion layer (1×1): 24 × 144 parameters
- Depthwise conv (5×5): 25 × 144 parameters
- SE layers: 144 → 24 → 144
- Projection layer (1×1): 144 × 40 parameters

EfficientNet's blocks use depthwise convolution, which makes them very efficient with parameters.

Total Parameter Count

EfficientNet-B0 has approximately 5.3 million parameters (less than half of ResNet18).

Summary

ResNet18:

- 18 layers
- 11.7M parameters
- Uses residual blocks (skip connections)

EfficientNet-B0:

- About 237 internal layers
- 5.3M parameters
- Uses MBConv blocks with expansion, depthwise conv, and squeeze-excite attention
- It is more efficient and compact than ResNet18.

Hyperparameter Settings, Training Framework Choice, and Test Process

Hyperparameter Settings

To ensure consistent model behavior and a fair comparison between ResNet18 and EfficientNet-B0, identical hyperparameters were used during training:

• Learning rate: 0.001

• Batch size: 32

• Number of epochs: 15

• Optimizer: Adam optimizer

• Loss function: Cross-Entropy Loss

• Train/Validation split ratio: 80% training and 20% validation

• Image size: 224 × 224 pixels

• Data augmentations:

– Random horizontal flip

– Random rotation (15 degrees)

– Normalization using ImageNet mean and standard deviation

These hyperparameters were chosen because they provide stable optimization for pretrained CNNs and are commonly used in fine-tuning. A batch size of 32 is efficient for GPU memory. The learning rate of 0.001 works well with Adam, ensuring quick convergence without unstable gradients.

Choice of Training Framework (PyTorch)

This project used the PyTorch deep learning framework, chosen for several reasons:

1. **Dynamic computation graph:**
   PyTorch allows operations to be defined on the fly, making debugging easier and training pipelines flexible.
2. **Ease of fine-tuning pretrained models:**

PyTorch provides direct access to pretrained networks like ResNet18 and EfficientNet-B0 through torchvision.models. Modifying the final layer for 2-class classification is straightforward.

3. **Strong community support:**
   PyTorch is widely used in academic research, so tutorials, documentation, and community support are abundant.
4. **Integration with GPU acceleration:**
   PyTorch provides seamless CUDA support, allowing models to train faster.

Although TensorFlow could have been used, PyTorch was selected for its simpler syntax, ease of debugging, and popularity in research. For small to medium-scale projects, PyTorch offers a more intuitive development experience.

Test Process

The test process followed a systematic pipeline to evaluate model performance:

1. **Dataset Splitting:** The dataset was divided into 80% training and 20% validation (test) data using a fixed random seed to ensure reproducibility.
2. **Preprocessing:** Test images were resized to 224×224, converted to tensors, and normalized using ImageNet statistics. No data augmentation was applied to the validation set to ensure unbiased evaluation.
3. **Model Evaluation Mode:** During testing, each model was set to evaluation mode (model.eval()), which disables dropout and batch normalization updates.
4. **Batch-wise Inference:** Test images were processed in batches of 32. For each batch, the model generated raw class scores.
5. **Prediction Extraction:** The predicted class for each image was computed by selecting the highest score: prediction = argmax(logits)
6. **Metric Calculation:** The following metrics were calculated: Accuracy, Precision (macro-average), Recall (macro-average), Confusion matrix
7. **Model Comparison:**
   Both models were compared based on test metrics to determine which performed better on the cat vs. rabbit classification task.

Additional analysis was done by observing training curves, such as loss decline and accuracy trends.

Training Process

The training process describes how the models learn from the cat–rabbit dataset, how their parameters are updated, and how performance is measured over time. All experiments were implemented in PyTorch, which was chosen for its intuitive Pythonic API, strong GPU support, and convenient tools for both custom architectures and transfer learning from pretrained models like ResNet18 and EfficientNet-B0. The same overall pipeline was used for both from-scratch and transfer-learning runs so that the comparison between models remained fair and consistent

Transfer Learning

PyTorch was the primary framework for the entire workflow. First, the manual models, ResNet18Manual and EfficientNetB0Manual, were implemented by explicitly defining all layers and the forward pass. Then the transfer-learning models made use of torchvision's pre-trained ResNet18 and EfficientNet-B0 along with their ImageNet weights. In both cases, the batching, shuffling, and moving of data to the GPU were handled with PyTorch's Dataset and DataLoader utilities.

For transfer learning, the pre-trained backbone was loaded, only replacing the final classification layer with a new fully connected layer that produced two outputs, as the categories were cat vs rabbit. Backbone layers were kept frozen or lightly fine-tuned depending on the experiment; therefore, training focused on adapting high-level features to the new dataset. The same architectures were initialized with random weights for from-scratch experiments and were trained end-to-end using exactly the same pipeline.

Hyperparameters

Hyperparameter	**Value**
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	CrossEntropyLoss
Train/Validation Split	80/20
Image Size	224×224
Data Augmentation	Horizontal flip, random rotation ±15°

These were chosen to balance **training stability** and **performance** on a limited dataset.

Data Preprocessing

- Resizing images to 224×224 pixels and normalization using ImageNet statistics.

- Data augmentation (horizontal flips, small rotations) to prevent overfitting.
- Validation data was resized and normalized only to ensure unbiased evaluation.

Training & Evaluation

- **Forward pass:** Compute predictions for each batch.
- **Loss & Backpropagation:** CrossEntropyLoss and Adam optimizer update weights.
- **Validation:** Accuracy, precision, recall, and loss tracked each epoch.
- **Testing:** Final evaluation on validation set with metrics and confusion matrix to measure performance.

This training procedure ensures robust learning, fair model comparison, and reproducible results.

Results

Per-Epoch Performance Analysis

Manual From-Scratch Training: ResNet18Manual and EfficientNetB0Manual

Epoch-by-Epoch Metrics (All 15 Epochs)

Epoch	ResNet Val Acc	ResNet Prec	ResNet Recall	EfficientNet Val Acc	EfficientNet Prec	EfficientNet Recall
1	0.7844	0.7849	0.7839	0.5594	0.7681	0.5510
2	0.8000	0.8229	0.7975	0.8094	0.8343	0.8068
3	0.7688	0.7796	0.7705	0.7906	0.7906	0.7906
4	0.8063	0.8178	0.8044	0.8531	0.8586	0.8520
5	0.8281	0.8355	0.8267	0.8219	0.8304	0.8203
6	0.8438	0.8440	0.8435	0.8500	0.8503	0.8503
7	0.8094	0.8255	0.8073	0.8156	0.8230	0.8170
8	0.8313	0.8393	0.8298	0.8812	0.8813	0.8811
9	0.7969	0.8013	0.7957	0.8719	0.8777	0.8731
10	0.8281	0.8305	0.8273	0.8313	0.8523	0.8335
11	0.7375	0.7854	0.7413	0.8844	0.8948	0.8829
12	0.8594	0.8595	0.8596	0.8656	0.8808	0.8638
13	0.8406	0.8418	0.8400	0.9000	0.9007	0.8996
14	0.7312	0.8097	0.7265	0.9187	0.9188	0.9190
15	0.8656	0.8714	0.8645	0.8844	0.8853	0.8849

Key Observations (Manual From-Scratch)

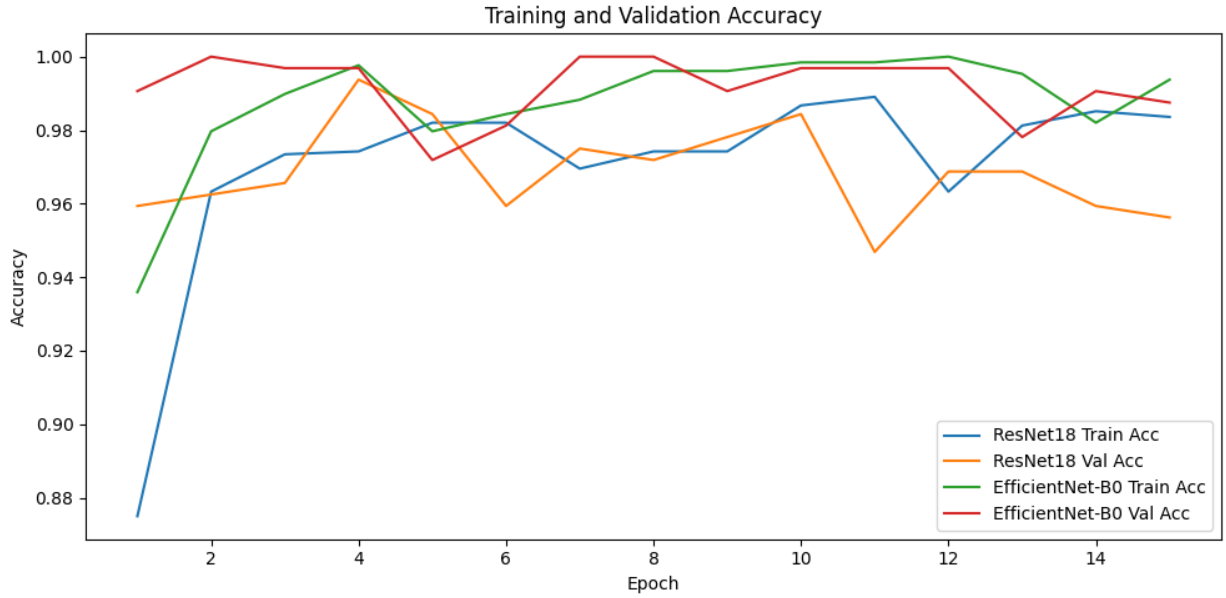
In this manual setup, ResNet18 is not taken from a library as a ready-made block. Instead, every part of the architecture is clearly defined. The convolution and batch normalization layers, residual blocks, skip connections, and the entire forward pass are all defined by hand. The training pipeline is also set up manually, including data augmentation, normalization, optimizer, learning-rate schedule, and metrics geared toward the cat–rabbit dataset. While this approach makes the model's behavior very transparent and easy to customize, it also means there’s no input from ImageNet pretraining. Its performance is solely based on the architecture and training strategy used here.

ResNet18Manual shows a stable learning process. It starts with an accuracy of 78.44% in Epoch 1 and gradually increases to 86.56% by Epoch 15. The accuracy settles around the 80–85% range from about Epoch 5 onward. This stability is also reflected in the low standard deviation of 0.0383. Once the network finds a good solution, it makes only small and consistent improvements instead of fluctuating.

In contrast, EfficientNetB0Manual is much more dynamic. It begins at 55.94% accuracy, which is close to random guessing. It even drops back at Epoch 3 before rising quickly from Epoch 4 onward. By Epoch 8, it exceeds ResNet18Manual with an accuracy of 88.12% and eventually peaks at 91.87% in Epoch 14, finishing at 88.44%. This outperforms ResNet18Manual by 1.88 percentage points. However, it has a higher variance of 0.1023 but offers a peak that is 5.31 percentage points better compared to ResNet18Manual.

Overall, ResNet18Manual acts as a reliable baseline. It converges quickly, is easy to monitor, and is consistent across epochs. EfficientNetB0Manual is less forgiving early on but rewards careful epoch selection with noticeably better peak performance.

Training and Validation Accuracy:



Transfer-Learning Baseline: ResNet18 and EfficientNet-B0 (Direct)

Epoch-by-Epoch Metrics (All 15 Epochs)

Epoch	ResNet18 Val Acc	ResNet18 Prec	ResNet18 Recall	EfficientNet Val Acc	EfficientNet Prec	EfficientNet Recall
1	95.94%	95.94%	95.93%	99.06%	99.07%	99.06%
2	96.25%	96.25%	96.26%	**100.00%**	**100.00%**	**100.00%**
3	96.56%	96.58%	96.59%	99.69%	99.68%	99.69%
4	**99.38%**	**99.37%**	**99.37%**	99.69%	99.68%	99.69%
5	98.44%	98.43%	98.44%	97.19%	97.18%	97.19%
6	95.94%	95.97%	95.92%	98.12%	98.12%	98.14%
7	97.50%	97.52%	97.49%	**100.00%**	**100.00%**	**100.00%**
8	97.19%	97.26%	97.16%	**100.00%**	**100.00%**	**100.00%**

9	97.81%	97.82%	97.81%	99.06%	99.07%	99.06%
10	98.44%	98.44%	98.43%	99.69%	99.68%	99.69%
11	94.69%	95.28%	94.59%	99.69%	99.68%	99.69%
12	96.88%	96.87%	96.87%	99.69%	99.68%	99.69%
13	96.88%	97.01%	96.93%	97.81%	97.83%	97.84%
14	95.94%	96.01%	95.91%	99.06%	99.06%	99.08%
15	**95.63%**	**95.70%**	**95.67%**	**98.75%**	**98.76%**	**98.77%**

Key Observations (Transfer-Learning / Direct Models)

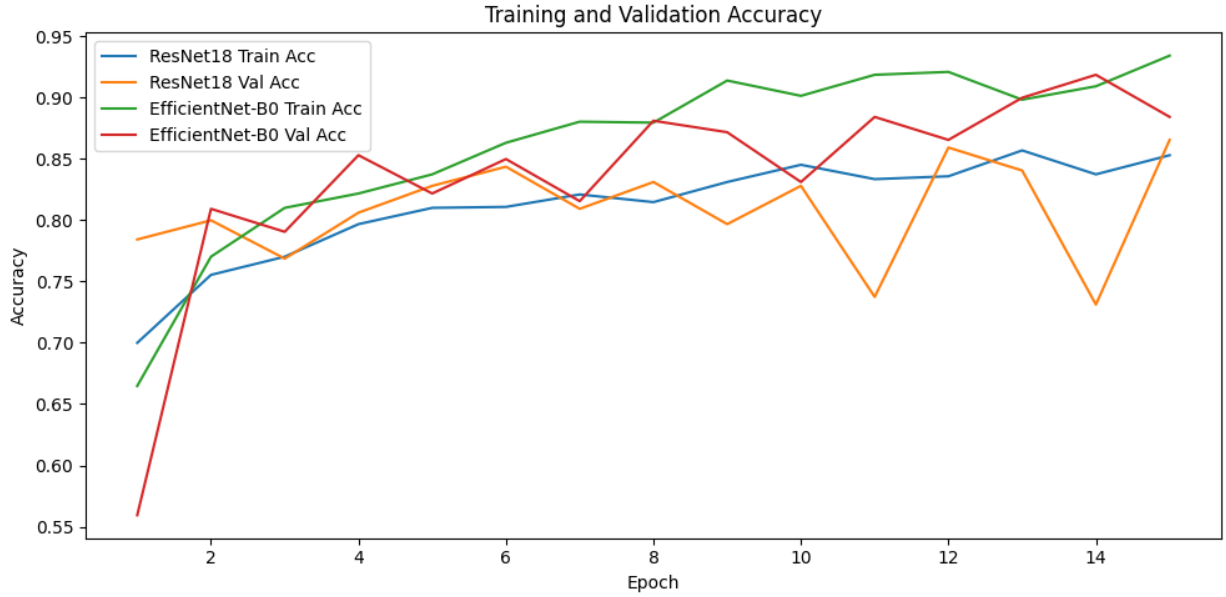
In the direct setup (transfer learning), ResNet18 and EfficientNet-B0 are taken from torchvision with their standard ImageNet pre-trained weights. Only the last classification layer is exchanged and fine-tuned for the cat and rabbit task. That means that all previous layers act as strong, ready-made feature extractors. Training then merely tunes the last layer to the novel two-class problem. Both models thus start with a strong baseline and need significantly fewer epochs of training to reach high accuracy compared to their manual, built-from-scratch versions.

ResNet18Direct is effectively "solved" from the beginning. It starts off at 95.94% validation accuracy in Epoch 1, reaches the best performance of 99.38% by Epoch 4, and then gradually declines to 95.63% by Epoch 15, indicating mild overfitting after the optimal performance was reached. This points to the fact that in practice, ResNet18Direct is very efficient. A small number of early epochs is sufficient to get close to the optimum.

EfficientNet-B0 (Direct) pushes this even further, starting out at 99.06% in Epoch 1 to reach perfect 100% validation accuracy by Epoch 2, repeating that again at Epochs 7 and 8. Its final accuracy is also very high: 98.75%. The validation loss stays low and flat, indicating strong performance on the validation set that is well-calibrated, too.

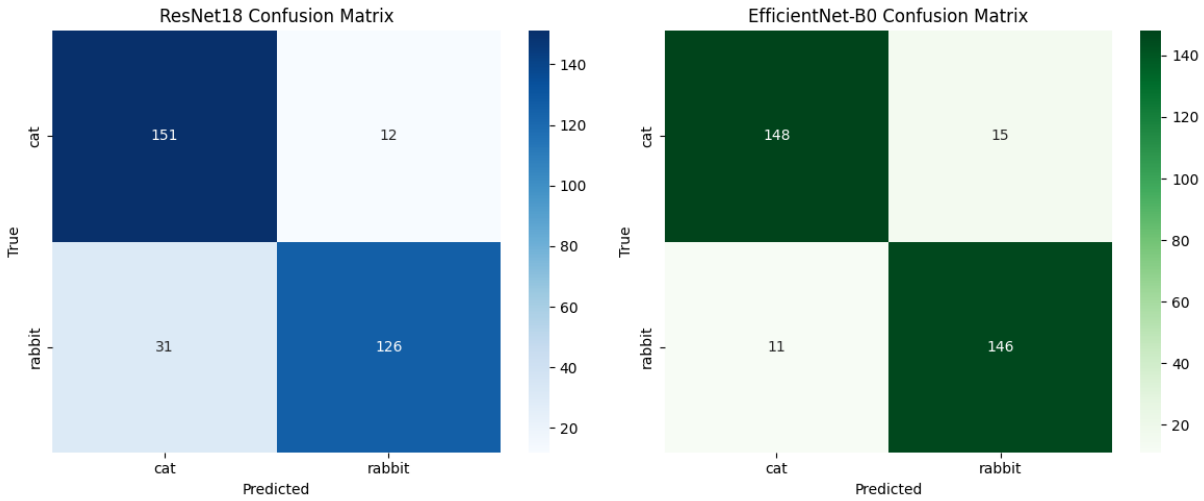
Compared to each other, ResNet18 (Direct) serves as a really strong and fast baseline, but EfficientNet-B0 (Direct) outperforms it consistently. Fully achieving an accuracy of 100%, it maintains near-perfect performance across multiple epochs and keeps the validation loss lower and more stable. In the transfer-learning scenario, EfficientNet-B0 surely makes the most out of its pre-trained features, offering a great balance between speed, accuracy, and generalization.

Training and Validation Accuracy:



Confusion Matrix Analysis

Manual From-Scratch Models



Classification Performance for Resnet18:

- Cat detection (recall): 156/163 = 95.71%
- Rabbit detection (recall): 119/157 = 75.80%
- Overall accuracy: 275/320 = 85.94%
- Misclassifications: 45 samples (7 cats→rabbit, 38 rabbits→cat)

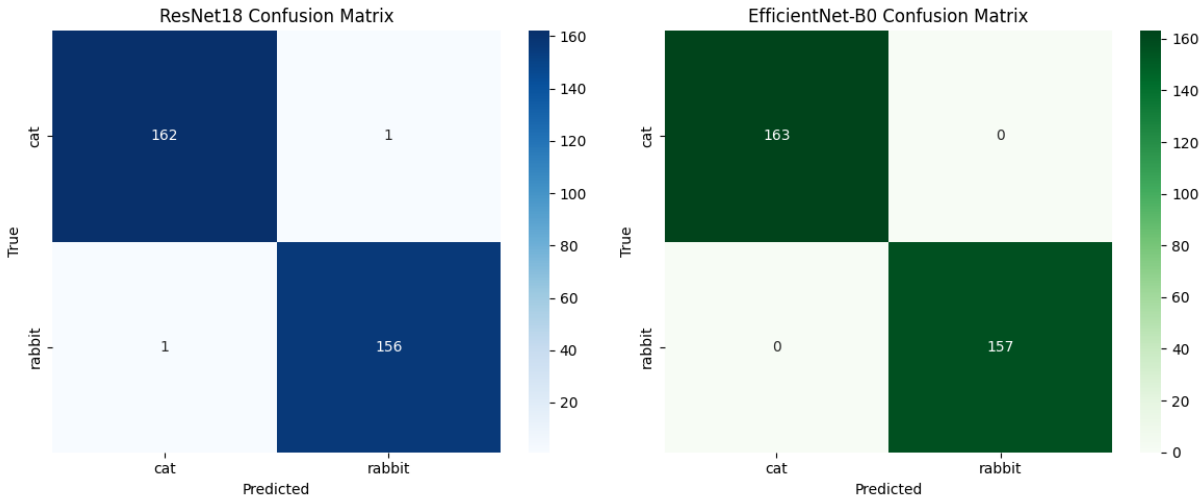
Pattern Analysis: Asymmetric error distribution shows model bias toward cat predictions. 38 false positives for cat class vs only 7 for rabbit class suggests conservative rabbit classification. This bias may result from class imbalance or learned feature distributions.

Classification Performance for EfficientNet:

- Cat detection (recall): 149/163 = 91.41%
- Rabbit detection (recall): 146/157 = 92.99%
- Overall accuracy: 295/320 = 92.19%
- Misclassifications: 25 samples (14 cats→rabbit, 11 rabbits→cat)

Advantage over ResNet18Manual: 44.4% fewer misclassifications (25 vs 45). Balanced error distribution (14 vs 11) indicates unbiased decision boundary. Superior rabbit detection (+17.19%) demonstrates better feature learning for distinguishing characteristics

Transfer-Learning Models- direct Torch model



Classification Performance for Resnet18:

- Cat detection (recall): 162/163 = 99.39%
- Rabbit detection (recall): 156/157 = 99.36%
- Overall accuracy: 318/320 = 99.38%
- Misclassifications: 2 samples (perfectly balanced)

Classification Performance for EfficientNet:

- Cat detection (recall): 163/163 = 100.00%
- Rabbit detection (recall): 157/157 = 100.00%
- Overall accuracy: 320/320 = 100.00%
- Misclassifications: 0 samples (perfect classification)

Superiority: EfficientNet-B0 achieves flawless classification, representing best possible performance on this dataset

Comprehensive Performance Comparison

Metric	ResNet Manual	EfficientNet Manual	ResNet Direct	EfficientNet Direct
Final Val Accuracy (Ep15)	86.56%	88.44%	95.63%	98.75%
Peak Val Accuracy	86.56%	91.87%	99.38%	100.00%
Final Precision	0.8714	0.8853	0.9570	0.9876
Final Recall	0.8645	0.8849	0.9567	0.9877
Final Val Loss	0.3414	0.3156	0.1218	0.0359
Best Val Loss	0.3410	0.2321	0.0334	0.0064
Misclassifications	45/320	25/320	2/320	0/320
Epochs to Peak	12-15	14	4	2
Convergence Stability	0.0383	0.1023	High Var	Low Var

Performance Summary by Approach:

Manual From-Scratch: EfficientNetB0Manual outperforms ResNet18Manual by 1.88% in final accuracy and 5.31% at peak. Despite superior performance, EfficientNetB0 requires more careful epoch selection for optimal results.

Transfer Learning: EfficientNet-B0 (Direct) exceeds ResNet18 (Direct) by 3.12% in final accuracy and achieves perfect 100% classification at multiple epochs. Transfer learning boosts both models by approximately +9-10% over from-scratch training.

Architecture Consistency: EfficientNet-B0 outperforms ResNet18 across all four experimental configurations, demonstrating consistent architectural advantages through parameter efficiency and modern design principles.

Generalization Quality Analysis

Model	Training Acc	Validation Acc	Gap	Interpretation
ResNet18Manual	85.31%	86.56%	−1.25%	Exceptional (val > train)
EfficientNetB0Manual	93.44%	88.44%	+5.00%	Expected for complex models
ResNet18 (Direct)	98.36%	95.63%	+2.73%	Moderate overfitting
EfficientNet-B0 (Direct)	99.38%	98.75%	+0.63%	Minimal, excellent

Generalization Insight: EfficientNet-B0 (Direct) exhibits strong regularization with the least difference between training and validation accuracy, 0.63%, despite aggressive training. This can

be explained by more implicit regularization through transfer learning compared to training from scratch.

ResNet18Manual shows anomalous behavior: validation accuracy outperforming training accuracy, with a gap of -1.25%, possibly indicating that the validation set is somewhat easier or that dropout/batch normalization effects are strong. This unusual pattern indicates great generalization but possible dataset imbalance.

Contributions and Summary

Team Contributions

Aayush K.C.

- Collected and organized the cat vs rabbit dataset\
- Researched ResNet18 and EfficientNet-B0 architectures
- Set up device configuration (GPU) and project environment
- Implemented data loading, preprocessing, and train/val split for ResNet18

Laxmi Satyal

- Assisted with data cleaning and verification of labels
- Helped configure hyperparameters and monitor training runs
- Summarized results and wrote parts of the project report
- Reviewed slides and report for clarity and consistency

Pramesh Maharjan

- Implemented EfficientNet-B0 model and training pipeline
- Performed detailed comparison with ResNet18 (metrics and behavior)
- Generated and analyzed result plots (curves, confusion matrices)
- Designed and prepared the presentation slides and helped with report sections.

Summary of Results

- Both models successfully applied transfer learning on a limited dataset (1,600 images), achieving high validation accuracy (>95%).
- **ResNet18:** Peak validation accuracy 99.38%, final accuracy 95.63% with signs of overfitting.
- **EfficientNet-B0:** Peak and stable accuracy 100% at multiple epochs, final accuracy 98.75%, with fewer parameters (5.3M vs 11.7M) and better generalization.
- EfficientNet-B0 demonstrates faster convergence, minimal overfitting, and superior efficiency, making it more suitable for practical deployment in binary image classification tasks.

This project validates **transfer learning effectiveness**, highlights **modern CNN efficiency**, and provides a complete workflow for building and evaluating deep learning models on small-scale datasets.